Engenharia de Software

4/4/2026

AT

Velocidade e Qualidade com Estruturas de Dados e Algoritmos

Professor(a): Ricardo Pires Mesquita

Samuel Hermany
INSTITUTO INFNET

Link com todas as questões

https://colab.research.google.com/drive/15v8L0KqyhQW5OkDE0ZV19Kc0r9DZeLj9?usp=drive_link

Link do GitHub

Com cada questão individual
https://github.com/faculdade-infnet/VI-1-velocidade-qualidade-com-algoritmos/tree/main/AT

Link do video

https://drive.google.com/file/d/1f3Nqeb1Y0r6jIqkq0lHTd_TcDv1sDDnl/view?usp=drive_link

1. QUESTÃO

A busca linear é um algoritmo simples que percorre o vetor inteiro até encontrar o valor ou chegar no final, então não precisa que os dados estejam organizados.

Observa-se que a busca binária é mais eficiente, mas só funciona quando o vetor está ordenado, porque ela vai dividindo o problema pela metade a cada passo. Para comparar os dois, foi colocado um contador de comparações, assim dá pra ver na prática a diferença de desempenho.

No geral, a busca linear cresce proporcional ao tamanho do vetor (O(n)), enquanto a binária cresce bem mais devagar (O(log n)).

Como a busca binária só funciona bem com o vetor ordenado, foi feita uma verificação antes de rodar o algoritmo.

Se o vetor não estiver ordenado, ele já para na hora (falha rápida), evitando resultado errado.

Essa verificação custa O(n), mas ainda assim é melhor do que ter que ordenar o vetor inteiro, que custaria O(n log n). Como a checagem é feita no vetor todo, não tem risco de erro.

No final, dá pra ver que a busca binária é bem mais eficiente, desde que essa condição inicial seja atendida. **Exemplos de saidas:**

```
---------------------------------------
Nº de comparações ao buscar (-14)
---------------------------------------
n=100      | linear=100        | binaria=6
n=1000     | linear=1000       | binaria=9
n=10000    | linear=10000      | binaria=13
n=100000   | linear=100000     | binaria=16
n=1000000  | linear=1000000    | binaria=19

Resultado da busca:
Nº -14, NÃO encontrado.

---------------------------------------
Nº de comparações ao buscar (11)
---------------------------------------
n=100      | linear=11         | binaria=3
n=1000     | linear=10         | binaria=10
n=10000    | linear=10000      | binaria=14
n=100000   | linear=100000     | binaria=17
n=1000000  | linear=1000000    | binaria=20

Resultado da busca:
Nº 11, encontrado.
```

2. QUESTÃO

Obs. Não fiz com 50mil e com 100mil, meu pc travava

Os algoritmos Bubble Sort e Selection Sort apresentaram comportamento O(n²) em todos os casos, com grande número de comparações independentemente do tipo de vetor, confirmando sua ineficiência para grandes volumes.

O Insertion Sort teve desempenho O(n) para vetores ordenados e quase ordenados, com poucas comparações e cópias, mas apresentou O(n²) nos casos reverso e aleatório.

A BST apresentou pior caso O(n²) para vetores ordenados e reversos, devido ao desbalanceamento da árvore, resultando em muitas comparações.

Para vetores quase ordenados e aleatórios, a BST teve desempenho próximo de O(n log n), com número bem menor de comparações em relação aos demais algoritmos.

Assim, observa-se que algoritmos simples são sensíveis à ordem dos dados, enquanto a BST pode ser eficiente, mas depende da estrutura da árvore.

```
===== TAMANHO: 1000 =====

--- Tipo: ordenado ---
Bubble: Comparações=499500, Cópias=0
Selection: Comparações=499500, Cópias=0
Insertion: Comparações=999, Cópias=999
BST: Comparações=499500, Chamadas=2001

--- Tipo: reverso ---
Bubble: Comparações=499500, Cópias=1498500
Selection: Comparações=499500, Cópias=1500
Insertion: Comparações=499500, Cópias=500499
BST: Comparações=499500, Chamadas=2001

--- Tipo: quase ---
Bubble: Comparações=499500, Cópias=82866
Selection: Comparações=499500, Cópias=150
Insertion: Comparações=28621, Cópias=28621
BST: Comparações=41946, Chamadas=2001

--- Tipo: aleatorio ---
Bubble: Comparações=499500, Cópias=753750
Selection: Comparações=499500, Cópias=2982
Insertion: Comparações=252242, Cópias=252249
BST: Comparações=10734, Chamadas=2001
```

```
===== TAMANHO: 10000 =====

--- Tipo: ordenado ---
Bubble: Comparações=49995000, Cópias=0
Selection: Comparações=49995000, Cópias=0
Insertion: Comparações=9999, Cópias=9999
BST: Comparações=49995000, Chamadas=20001

--- Tipo: reverso ---
Bubble: Comparações=49995000, Cópias=149985000
Selection: Comparações=49995000, Cópias=15000
Insertion: Comparações=49995000, Cópias=50004999
BST: Comparações=49995000, Chamadas=20001

--- Tipo: quase ---
Bubble: Comparações=49995000, Cópias=9517314
Selection: Comparações=49995000, Cópias=1500
Insertion: Comparações=3182437, Cópias=3182437
BST: Comparações=873924, Chamadas=20001

--- Tipo: aleatorio ---
Bubble: Comparações=49995000, Cópias=76528947
Selection: Comparações=49995000, Cópias=29973
Insertion: Comparações=25519641, Cópias=25519648
BST: Comparações=147042, Chamadas=20001
```

```
===== TAMANHO: 25000 =====

--- Tipo: ordenado ---
Bubble: Comparações=312487500, Cópias=0
Selection: Comparações=312487500, Cópias=0
Insertion: Comparações=24999, Cópias=24999
BST: Comparações=312487500, Chamadas=50001

--- Tipo: reverso ---
Bubble: Comparações=312487500, Cópias=937462500
Selection: Comparações=312487500, Cópias=37500
Insertion: Comparações=312487500, Cópias=312512499
BST: Comparações=312487500, Chamadas=50001

--- Tipo: quase ---
Bubble: Comparações=312487500, Cópias=60149730
Selection: Comparações=312487500, Cópias=3750
Insertion: Comparações=20074909, Cópias=20074909
BST: Comparações=3510242, Chamadas=50001

--- Tipo: aleatorio ---
Bubble: Comparações=312487500, Cópias=469058826
Selection: Comparações=312487500, Cópias=74964
Insertion: Comparações=156377932, Cópias=156377941
BST: Comparações=451764, Chamadas=50001
```

3. QUESTÃO

O tempo total aumenta quando o array cresce, mas não é totalmente linear por causa da ordenação da versão A.

Tirar os duplicados foi rápido, cresce mais ou menos O(n), mesmo com array grande.

A versão A (ordenar tudo) demora mais, tipo O(n log n), fica lento quando o array sem duplicados é grande.

A versão B (QuickSelect) é bem mais rápida, cresce quase linear O(n), mesmo nos maiores arrays.

Comparando os dois, QuickSelect ganha fácil, principalmente quando k é bem menor que n.

A diferença de tempo fica maior quanto maior é o array sem duplicados.

Sort completo só vale a pena se você precisa de todos os elementos ordenados.

No fim, QuickSelect é mais eficiente e ainda acerta os k menores sem erro.

Tamanho original	Tamanho sem duplicados	Remoção duplicados (s)	Sort (A) (s)	QuickSelect (B) (s)	Tempo total (s)
1000	435	0.00091	0.00051	0.00010	0.00250
10000	4311	0.01032	0.00577	0.00064	0.02541
25000	108140	0.01993	0.01758	0.00221	0.05678
50000	21646	0.10542	0.03681	0.00390	0.17717
100000	43265	0.16957	0.0763	0.00592	0.31501

```
===== TAMANHO: 1000 =====
Remoção duplicados: 0.00091s | tamanho final: 435
Sort (A):           0.00051s
QuickSelect (B):    0.00010s
Tempo total:        0.00250s
Iguais? True

===== TAMANHO: 10000 =====
Remoção duplicados: 0.01032s | tamanho final: 4311
Sort (A):           0.00577s
QuickSelect (B):    0.00064s
Tempo total:        0.02541s
Iguais? True

===== TAMANHO: 25000 =====
Remoção duplicados: 0.01993s | tamanho final: 10810
Sort (A):           0.01758s
QuickSelect (B):    0.00221s
Tempo total:        0.05678s
Iguais? True

===== TAMANHO: 50000 =====
Remoção duplicados: 0.10542s | tamanho final: 21646
Sort (A):           0.03681s
QuickSelect (B):    0.00390s
Tempo total:        0.17717s
Iguais? True

===== TAMANHO: 100000 =====
Remoção duplicados: 0.16957s | tamanho final: 43265
Sort (A):           0.07630s
QuickSelect (B):    0.00592s
Tempo total:        0.31501s
Iguais? True
```

4. QUESTÃO

Nesse exercício eu fiz uma tabela hash usando lista ligada em cada bucket.
O put coloca valor ou atualiza se a chave já existir, get procura a chave e delete tira a chave.
A tabela tem rehash automatico, ou seja, quando passa do limite de carga (load_factor = 0,7) ela aumenta o tamanho e tenta manter tudo que já tinha.
No teste eu coloquei 10 elementos numa tabela bem pequena (capacity=2) pra forçar o rehash.
As comparações ficaram assim:

Operação	Comparações	Fator de carga (α)	Custo esperado O(1+α)
Inserção 10 elementos	8	0.62	O(1 + 0.62) ≈ O(1.62)
Busca 10 elementos	20	0.62	O(1 + 0.62) ≈ O(1.62)

Cada comparação é tipo “olhar cada nó da lista pra ver se a chave tá lá”.

Depois do rehash, o fator de carga ficou 0,62 porque a tabela dobrou de tamanho.

O custo esperado de O(1+α) deu mais ou menos 1,62 comparações por operação, que bateu com o que apareceu nos testes.

Então parece que a tabela funciona certinho, mantém as chaves depois do rehash e o número de comparações cresce conforme o fator de carga, como era esperado.

```
Comparações na inserção de 10 elementos: 8
Comparações na busca de 10 elementos: 20
Fator de carga atual: 0.62
Custo esperado de busca e inserção é O(1 + α), ou seja:
O(1 + 0.62) ≈ O(1.62) comparações por operação
```

5. QUESTÃO

Análise do esforço e observações

O DFS recursivo mostra a árvore de forma “desenhada”, com indentação por nível, então dá pra ver quem tá onde na estrutura.

O DFS com pilha percorre os caminhos mais profundos primeiro, então é útil pra labirintos ou quando você precisa explorar tudo antes de voltar.

O BFS visita a árvore nível por nível, sendo bom pra encontrar elementos próximos da raiz de forma rápida.

A contagem de comparações no BFS indica quantos nodos foram visitados.

A inserção na BST normalmente é rápida (O(log n)), mas se a árvore ficar desbalanceada pode piorar e chegar a O(n).

```
Elementos inseridos: [17, 88, 5, 19, 69, 58, 48, 42, 38, 46]

DFS recursivo (com indentacao):
17
  5
  88
    19
      69
        58
          48
            42
              38
              46

DFS com pilha (profundidade): [17, 5, 88, 19, 69, 58, 48, 42, 38, 46]
BFS com fila (largura): [17, 5, 88, 19, 69, 58, 48, 42, 38, 46]
Numero de comparacoes BFS: 10
```

6. QUESTÃO

Fiz uma lista encadeada simples, cada nó é uma caixinha que aponta pro próximo. Tem métodos pra colocar no começo, no fim, em qualquer posição, procurar valor e remover valor ou posição. Também fiz __len__ pra ver tamanho e __str__ pra mostrar a lista.

Pra inserção/remover por posição tem checagem de índice e mensagem de erro, e se não achar valor, também avisa.

Big O das operações tá resumido na tabela abaixo:

Operação	Custo esperado	Observações rápidas
inserir_no_inicio(valor)	O(1)	só muda ponteiro do começo
inserir_no_fim(valor)	O(n)	precisa andar até o último nó
inserir_na_posicao(indice,valor)	O(n)	tem que andar até a posição
buscar(valor)	O(n)	percorre até achar
remover(valor)	O(n)	percorre até achar
remover_na_posicao(indice)	O(n)	anda até posição certa
len()	O(1)	devolve contador rápido
str()	O(n)	percorre todos os nós pra mostrar

```
10 -> 15 -> 20 -> $
Valor 15 encontrado na posição 1
Valor 20 removido
Nó na posição 0 removido
15 -> $
Tamanho: 1
```

7. QUESTÃO

Aqui eu implementei uma lista duplamente encadeada e depois um deque usando essa lista. Testei inserções e remoções alternadas nas duas pontas, e depois conferimos se os ponteiros e o tamanho estavam corretos (as chamadas de invariantes).

Sequência de operações e resultados:

Operação	Lista atual	Resultado da verificação de invariantes
Inserir 10 esquerda	[10]	Tudo certo!
Inserir 20 direita	[10, 20]	Tudo certo!
Inserir 5 esquerda	[5, 10, 20]	Tudo certo!
Inserir 25 direita	[5, 10, 20, 25]	Tudo certo!
Espiar esquerda	-	5
Espiar direita	-	25
Remover esquerda	[10, 20, 25] → remove 5	Tudo certo!
Remover direita	[10, 20] → remove 25	Tudo certo!
Inserir 30 direita	[10, 20, 30]	Tudo certo!
Inserir 2 esquerda	[2, 10, 20, 30]	Tudo certo!
Inserir 40 direita	[2, 10, 20, 30, 40]	Tudo certo!
Tamanho final	-	5

```
Depois das inserções:
Lista:  [5, 10, 20, 25]
Verificando invariantes...
Tudo certo! Ponteiros consistentes e tamanho correto.
Espiar esquerda: 5
Espiar direita: 25
Remover esquerda: 5
Remover direita: 25
Depois das remoções:
Lista:  [10, 20]
Verificando invariantes...
Tudo certo! Ponteiros consistentes e tamanho correto.
Inserções extras:
Lista:  [2, 10, 20, 30, 40]
Verificando invariantes...
Tudo certo! Ponteiros consistentes e tamanho correto.
Tamanho final do deque: 5
```

8. QUESTÃO

Nesse exercício eu fiz o problema da mochila 0/1 usando duas formas, uma só com recursão e outra usando memoization com hash. O objetivo era pegar os itens com maior valor possível sem passar do peso máximo.

Na versão recursiva pura funcionou certo e deu o valor máximo 225, porém teve muitas chamadas, no total 13897. Isso acontece porque o código fica repetindo várias contas que já fez antes, então acaba ficando meio pesado.

Também contei os subproblemas diferentes e deu 275, então dá pra ver que não eram tantos casos assim, mas mesmo assim ele repetiu bastante coisa.

Depois fiz com memoization usando a HashTable da outra questão. O resultado continuou o mesmo (225), mas o número de chamadas caiu bastante, foi para 449. Então deu pra ver que melhorou bem.

Tipo	Valor	Chamadas	Subproblemas
Recursivo	225	13897	275
Com memoization	225	449	-

A chave que usei foi (i, capacidade), porque isso já define o problema naquele momento, tipo qual item estou olhando e quanto ainda cabe.

Sobre o tempo de execução, o recursivo é bem mais lento, cresce muito rápido (tipo 2^n), já com memoization fica melhor, mais ou menos n * W, porque ele não repete as contas.

No final deu pra ver que usar hash ajudou bastante e deixou o código bem mais rápido.

```
RESULTADO MOCHILA 0/1
-----------------------------------
Recursivo puro:
Valor máximo: 225
Chamadas: 13897
Subproblemas diferentes: 275

Com memoization (HashTable):
Valor máximo: 225
Chamadas: 449
```